

Binary Tree-Inorder ▾

## BINARY TREE- INORDER REVERSAL

Write a C program to create a Binary Tree and perform Recursive Inorder Traversal.

Consider an organizational hierarchy management system where employee records are stored in a

```
2 #include <stdlib.h>
3
4 struct Node
5 {
6     int data;
7     struct Node *left;
8     struct Node *right;
9 };
10
11 struct Node* createNode(int data)
12 {
13     struct Node *newNode = malloc(sizeof(struct Node));
14     newNode->data = data;
15     newNode->left = NULL;
16     newNode->right = NULL;
17     return newNode;
18 }
19
20 struct Node* insert(struct Node *root, int data)
```

```
45     struct Node *root = NULL;
46
47     root = insert(root, 50);
48     insert(root, 30);
49     insert(root, 70);
50     insert(root, 20);
51     insert(root, 40);
52     insert(root, 60);
53     insert(root, 80);
54
55     printf("Inorder Traversal: ");
56     inorder(root);
57
58     return 0;
59 }
```

## OUTPUT

```
Inorder Traversal: 20 30 40 50 60 70 80
```

BT- Rec. Preorder T ▼

## BT- REC. PREORDER TRAVERSE

Write a C program to create a Binary Tree and perform Recursive Preorder Traversal.

Consider a file backup application where directories and subdirectories are represented as a tree structure. The program ▼

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node *left, *right;
7 };
8
9 struct Node* create(int data) {
10     struct Node *newNode = malloc(sizeof(struct Node));
11     newNode->data = data;
12     newNode->left = NULL;
13     newNode->right = NULL;
14     return newNode;
15 }
16
17 struct Node* insert(struct Node *root, int data) {
18     if (root == NULL)
19         return create(data);
20 }
```

```
38     struct Node *root = NULL;
39
40     root = insert(root, 10);
41     insert(root, 5);
42     insert(root, 15);
43     insert(root, 3);
44     insert(root, 7);
45     insert(root, 12);
46     insert(root, 18);
47
48     printf("Preorder Traversal: ");
49     preorder(root);
50
51     return 0;
52 }
```

## OUTPUT

```
Preorder Traversal: 10 5 3 7 15 12 18
```

BT- Rec. Postorder ▾

## BT- REC. POSTORDER TRAVERSE

Write a C program to create a Binary Tree and perform Recursive Postorder Traversal. Consider a software uninstallation utility where dependent components must be removed before the parent application. The ▾

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node *left, *right;
7 };
8
9 struct Node* create(int data) {
10     struct Node *newNode = malloc(sizeof(struct Node));
11     newNode->data = data;
12     newNode->left = NULL;
13     newNode->right = NULL;
14     return newNode;
15 }
16
17 struct Node* insert(struct Node *root, int data) {
18     if (root == NULL)
19         return create(data);
20 }
```

```
39
40     root = insert(root, 25);
41     insert(root, 15);
42     insert(root, 35);
43     insert(root, 10);
44     insert(root, 20);
45     insert(root, 30);
46     insert(root, 40);
47
48     printf("Postorder Traversal: ");
49     postorder(root);
50
51     return 0;
52 }
```

## OUTPUT

```
Postorder Traversal: 10 20 15 30 40 35 25
```

BST- In,Pre,Post Tra ▼

## BST- IN,PRE,POST TRAVERSE

Write a C program to create a Binary Search Tree (BST) and perform Inorder, Preorder, and Postorder Traversals. Consider a student record management system where roll numbers are stored in a BST and different ▼

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node *left, *right;
7 };
8
9 struct Node* create(int data) {
10     struct Node *n = malloc(sizeof(struct Node));
11     n->data = data;
12     n->left = n->right = NULL;
13     return n;
14 }
15
16 struct Node* insert(struct Node *root, int data) {
17     if (root == NULL)
18         return create(data);
```

```
62
63     printf("Inorder: ");
64     inorder(root);
65
66     printf("\nPreorder: ");
67     preorder(root);
68
69     printf("\nPostorder: ");
70     postorder(root);
71
72     return 0;
73 }
```

## OUTPUT

```
Inorder: 15 25 35 45 55 65 75
Preorder: 45 25 15 35 65 55 75
Postorder: 15 35 25 55 75 65 45
```



BST- Search Element ▼

## BST- SEARCH ELEMENT

Write a C program to insert elements into a Binary Search Tree and search for a specific element. Consider an online product catalog where product IDs are stored in a BST for efficient retrieval. The program should

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node *left, *right;
7 };
8
9 struct Node* create(int data) {
10     struct Node *n = malloc(sizeof(struct Node));
11     n->data = data;
12     n->left = n->right = NULL;
13     return n;
14 }
15
16 struct Node* insert(struct Node *root, int data) {
17     if (root == NULL)
18         return create(data);
19
20     if (data < root->data)
```

```
44     root = insert(root, 50);
45     insert(root, 30);
46     insert(root, 70);
47     insert(root, 20);
48     insert(root, 40);
49     insert(root, 60);
50     insert(root, 80);
51
52     if (search(root, 60))
53         printf("Element 60 Found in the Binary Search Tree.");
54     else
55         printf("Element 60 Not Found in the Binary Search Tree.");
56
57     return 0;
58 }
```

## OUTPUT

```
Element 60 Found in the Binary Search Tree.
```

BST- Finding Height ▾

## BST- FINDING HEIGHT

Write a C program to find the height of a Binary Tree. Consider a network routing structure where devices are arranged hierarchically and the tree height represents the maximum communication depth. ▾

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node *left, *right;
7 };
8
9 struct Node* create(int data) {
10     struct Node *n = malloc(sizeof(struct Node));
11     n->data = data;
12     n->left = n->right = NULL;
13     return n;
14 }
15
16 struct Node* insert(struct Node *root, int data) {
17     if (root == NULL)
18         return create(data);
```

```
41     struct Node *root = NULL;
42
43     root = insert(root, 1);
44     insert(root, 2);
45     insert(root, 3);
46     insert(root, 4);
47     insert(root, 5);
48     insert(root, 6);
49     insert(root, 7);
50
51     printf("Height of Binary Tree: %d", height(root));
52
53     return 0;
54 }
55
```

## OUTPUT

```
Height of Binary Tree: 7
```

BST- Count Nodes ▾

## BST- COUNT NODES

Write a C program to count the total number of nodes, leaf nodes, and non-leaf nodes in a Binary Tree. Consider a company's organizational structure where leaf nodes represent employees without

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node *left, *right;
7 };
8
9 struct Node* create(int data) {
10     struct Node *n = malloc(sizeof(struct Node));
11     n->data = data;
12     n->left = n->right = NULL;
13     return n;
14 }
15
16 struct Node* insert(struct Node *root, int data) {
17     if (root == NULL)
18         return create(data);
19 }
```

```
58     insert(root, 30);
59     insert(root, 70);
60     insert(root, 20);
61     insert(root, 40);
62     insert(root, 60);
63     insert(root, 80);
64
65     printf("Total Nodes: %d\n", totalNodes(root));
66     printf("Leaf Nodes: %d\n", leafNodes(root));
67     printf("Non-Leaf Nodes: %d\n", nonLeafNodes(root));
68
69     return 0;
70 }
```

## OUTPUT

```
Total Nodes: 7
Leaf Nodes: 4
Non-Leaf Nodes: 3
```

BST- Min, Max Valu ▾

## BST- MIN, MAX VALUES

Write a C program to find the minimum and maximum values in a Binary Search Tree. Consider a sales database where transaction amounts are stored in a BST and management wants to identify the

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node *left, *right;
7 };
8
9 struct Node* create(int data) {
10     struct Node *n = malloc(sizeof(struct Node));
11     n->data = data;
12     n->left = n->right = NULL;
13     return n;
14 }
15
16 struct Node* insert(struct Node *root, int data) {
17     if (root == NULL)
18         return create(data);
19
20     if (data < root->data)
```

```
42
43     root = insert(root, 100);
44     insert(root, 50);
45     insert(root, 150);
46     insert(root, 25);
47     insert(root, 75);
48     insert(root, 125);
49     insert(root, 175);
50
51     printf("Minimum Value: %d\n", minValue(root));
52     printf("Maximum Value: %d\n", maxValue(root));
53
54     return 0;
55 }
```

## OUTPUT

```
Minimum Value: 25
Maximum Value: 175
```



```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node *left, *right;
7 };
8
9 struct Node* create(int data) {
10     struct Node *n = malloc(sizeof(struct Node));
11     n->data = data;
12     n->left = n->right = NULL;
13     return n;
14 }
15
16 struct Node* insert(struct Node *root, int data) {
17     if (root == NULL)
18         return create(data);
19
20     if (data < root->data)
```

```
74     root = insert(root, 50);
75     insert(root, 30);
76     insert(root, 70);
77     insert(root, 20);
78     insert(root, 40);
79     insert(root, 60);
80     insert(root, 80);
81
82     root = deleteNode(root, 30);
83
84     printf("Inorder Traversal after Deletion: ");
85     inorder(root);
86
87     return 0;
88 }
```

## OUTPUT

```
Inorder Traversal after Deletion: 20 40 50 60 70 80
```

**BST- MIRRORING**

Write a C program to mirror a Binary Tree and display the inorder traversal before and after mirroring. Consider a graphical application where tree structures need to be reflected horizontally for

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node
5 {
6     int data;
7     struct Node *left, *right;
8 };
9
10 struct Node* create(int data)
11 {
12     struct Node *n = malloc(sizeof(struct Node));
13     n->data = data;
14     n->left = n->right = NULL;
15     return n;
16 }
17
18 struct Node* insert(struct Node *root, int data)
19 {
20     if (root == NULL)
```

```
64     insert(root, 7);
65     insert(root, 12);
66     insert(root, 18);
67
68     printf("Original Inorder: ");
69     inorder(root);
70
71     mirror(root);
72
73     printf("\nMirrored Inorder: ");
74     inorder(root);
75
76     return 0;
77 }
```

## OUTPUT

```
Original Inorder: 3 5 7 10 12 15 18
Mirrored Inorder: 18 15 12 10 7 5 3
```